Leveraging Advanced Machine Learning Algorithms for Loan Repayment Prediction

Akshat Ranjan
School of Computer Engineering
KIIT Deemed to be University
Bhubaneshwar, Odisha, India
akshatranjan000@gmail.com

Mahendra Kumar Gourisaria
School of Computer Engineering
KIIT Deemed to be University
Bhubaneswar, India
mkgourisaria2010@gmail.com

Jay Prakash Singh
Dept of Computer Science & Engineering
Manipal University Jaipur
Jaipur, Rajasthan, India
jaykiit.research@gmail.com

Amiya Ranjan Panda
School of Computer Engineering
KIIT Deemed to be University
Bhubaneshwar, Odisha, India
amiya.pandafcs@kiit.ac.in

Soumya Ranjan Mishra
School of Computer Engineering
KIIT Deemed to be University
Bhubaneswar, Odisha, India
soumyaranjanmishra.in@gmail.com

Varutri Parihar
School of Computer Engineering
KIIT Deemed to be University
Bhubaneswar, India
varutriparihar@gmail.com

2024 15th International Conference on Computing Communication and Networking Technologies (ICCCNT) | 979-8-3503-7024-9/24/$31.00 ©2024 IEEE | DOI: 10.1109/ICCCNT61001.2024.10724308

***Abstract*—Accurately predicting loan repayment behavior is a critical challenge for financial institutions, which often face high default rates and financial instability due to inaccurate credit assessments. In order to address these problems, this study looks into the application of sophisticated machine learning methods such Random Forests, Decision Trees, Support Vector Machines, and Stacking Classifiers. The models, trained on diverse client variables, significantly outperform traditional credit scoring methods. The findings suggest that integrating these sophisticated models can enhance financial performance, reduce default risks, and enable more informed lending decisions. The study also examines potential avenues for future research, including the application of increasingly sophisticated machine learning techniques and the integration of new data sources to enhance models. This study advances the field of machine learning in finance by providing business and academics with insightful information that could result in stronger and more secure financial systems.**

Keywords—Loan Repayment, Machine Learning, Classification, Stacking Classifier, Credit Score

I. INTRODUCTION

The arrival of machine learning (ML) technology has had a profound impact on the financial landscape. Predicting loan repayment behaviour is one area where machine learning has demonstrated enormous promise. This is a crucial component of both lenders' and borrowers' financial well-being [1-3]. Currently, machine learning is being widely used in the financial institutions for applications like financial advisory to investors, automated trading bots, and fraud detection. This research explores how lenders' loan repayment behaviour can be predicted using classification algorithms. Repaying a loan is a complicated process that is impacted by a wide range of variables, from macroeconomic considerations like interest rates and economic growth to personal situations like employment status and income level. Borrowers who fail to return their debts may face severe financial repercussions, such as lawsuits, credit score ruin, and prolonged financial hardship. Due to defaults and non-performing loans, lenders may suffer large financial losses. For this reason, it is crucial to forecast loan repayment behaviour properly for both parties [4].

Numerous investigations have looked into what influences debt repayment. The loan repayment performance of individuals is performance of individuals is greatly affected by the loan repayment period/term, the grace period (if any), and the timeliness of loan release. Usually, poor loan repayment performance results from a delay in the release of loan funds and cumbersome loan application and disbursement procedures on the bank's side. The studies mentioned above underscore the need to comprehend the diverse aspects that may impact the repayment behaviour of loans. In the post-pandemic period, the likelihood of loan default rose from 0.056 to 0.079. In recent years, ML has emerged as a powerful tool for predicting loan repayment behaviour. It can be leveraged as a replacement for traditional credit score techniques, as ML has the ability to sift through unprocessed data in large amounts and identify better and more complex patterns using algorithms like K-Nearest Neighbours, Decision Trees, Random Forest etc.

Although, depending on the particular context and data employed, these models' efficacy may differ. Consequently, additional investigation and verification are required to guarantee their suitability in various contexts. With an emphasis on the Indian context, this paper explores the application of machine learning (ML) in predicting loan repayment behaviour in an effort to add to the increasing body of research in this area. The rest of the paper is divided in the following section. Section II Related work followed by Section III methods and materials. The Section IV talks about classification models which we have ed the different classifier used the implementation. Finally implementation and results mentioned in section V followed by conclusion and future work in section VI.

II. RELATED WORK.

The field of loan repayment behavior analysis is getting a major contribution from various machine learning research. Y. Liang et al. [1] demonstrate the use of machine learning to predict loan repayment behavior by utilizing algorithms like Random Forest, Logistic Regression, Naïve Bayes, LightGBM and Neural Networks. They have used up-sampling and down-sampling techniques and achieved the best accuracy with the Logistic Regression model with a value of 0.6934. Three feature selection techniques—Information Gain, Gini Test, and Chi-Squared Test—as well as machine learning algorithms—Decision Tree, Logistic Regression, and Random Forest—were employed in a study by A. Mueankoo et al. [5]. The Decision Tree model yielded the highest accuracy of 0.631.

In his study [6], C. Han, a number of machine learning models are trained using data from Lending Club to assess if the borrower has the capacity to repay the loan. They have employed different machine learning algorithm. The models were tested using ROC curves, and Random Forest produced the best accuracy (0.70407) and Logistic Regression produced the highest AUC score (0.7015). A paper by A. Sarkar [7] used Random Forest, Decision Tree, and Logistic Regression in another study, and the Logistic Regression model produced the best accuracy of 0.8078.

A paper by Y. Diwate et al. explores loan approval rate prediction using machine learning. The model that they have used is Support Vector Machine and achieved a precision of 0.811 with it. The result could have been better with the use of more classifiers [8]. M. Madaan et al. [9] in their paper used two algorithms – Random Forest and Decision Trees with Random Forest giving a much greater accuracy compared to Decision Trees with a value of 0.80.

III. Methods And Materials

This section discusses the features of the dataset, and the methods used to clean and analyze it. It has been broken up into three subsections – *A. Dataset Description, B. Dataset Pre-Processing, C. Methods Description.*

A. Dataset Description

The dataset has been taken from Kaggle [10] and contains a wide amount of information about borrowers who have taken loans from a bank. The information includes features like the person's salary, loan amount, loan term, person's recent bankruptcies, etc. These features can be extremely helpful in predicting a person's ability to pay back a loan on time. The dataset contained three categories for the target variable i.e. the "loan_status" column, namely, Charged Off, Current, and Fully Paid. The dataset had 5627 entries in the "Charged Off", 1140 entries in the "Current", and 32950 entries in the "Fully Paid" categories.

B. Dataset Pre-processing

Cleaning the data for optimizing the workflow is an essential step in developing an efficient model. In this section, we will discuss, how we cleaned our dataset. The section is divided into the following subsections: *1) Null Value Removal, 2) Imputing Missing Values, 3) Label Encoding, 4) Fixing Temporal Variables, 5) Data Balancing, 6) Correlation heatmap*

1) *Null Value Treatment:* The dataset contained 54 completely null valued columns and 4 columns with more than 20% null values, making it unnecessarily large. To solve this problem, we dropped all the columns with more than 20% null values.

2) *Imputing Missing Values:* For features with less than 20% missing values, we used the SimpleImputer class from scikit-learn.

3) *Label Encoding:* The dataset also contains a large number of categorical features. So, to convert these into numerical features, we use the LabelEncoder class from scikit-learn.

4) *Fixing Temporal Variables:* The dataset also contains a few temporal variables namely - "issue_d", "earliest_cr_line", "last_pymnt_d", and "last_credit_pull_d".

To convert them to numerical features, we subtract their values from their minimum values to get the number of days instead of a date.

5) *Data Balancing:* The data was also highly imbalanced. This can create a bias while training the models. To solve this, we up-sampled the data by using the SMOTETomek class from scikit-learn. In the end we have reduced the number of columns from 111 in the original dataset to 53 in the modified dataset.

6) *Correlation Heatmap:* We also plot a heatmap for the correlation values to find which features are highly correlated which are mentioned in Fig. 1.

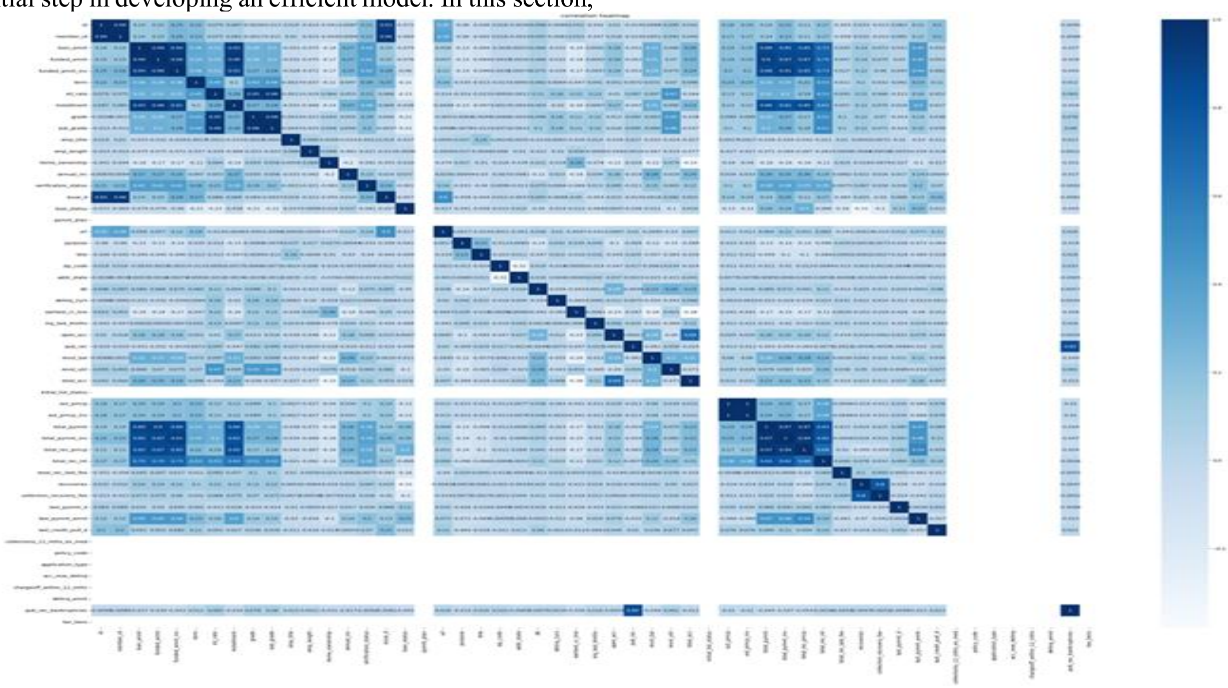

Fig. 1. Correlation Heatmap

C. Method Description

In this experiment, we used various machine learning algorithms, each with their own unique parameters. They are as follows: Logistic Regression, Decision Tree, K-Nearest Neighbors, Support Vector Machine, Naive Bayes, Random Forest, Ada Boost, Gradient Boosting, Voting Classifier, Stacking Classifier. We have already split out dataset into 80% training and 20% testing data using TrainTestSplit from scikit-learn.

IV. Classification Model

A. Logistic Regression (LR)

The method of modeling the probability of a discrete result given an input variable is known as logistic regression. In most cases, the results of logistic regression models are binary, meaning that they can have two possible values, such as true or false, yes or no, and so on. A practical method for modeling events with more than two distinct discrete outcomes is multinomial logistic regression. A helpful analysis technique for classification issues is logistic regression, which is applied to ascertain which group a fresh sample most closely fits into. [11]. This statistical technique examines the relationship between two data elements. When doing binary classification using logistic regression, a sigmoid function is utilized. This function receives input as independent variables and outputs a probability value between 0 and 1. The logistic function is given in equation 1.

$$p(x) = \frac{1}{1 + e^{-(\beta_0 + \beta_1 x)}} \tag{1}$$

B. Decision Tree (DT)

Another classification model that is very widely used is Decision Trees. It builds a model of data-driven judgments that resembles a tree. Data is recursively partitioned according to attribute values in order to create a decision tree [12]. Fig. 2 illustrates the mechanism of a decision tree. A measure like entropy or Gini impurity is used to determine which characteristic is optimal for a split. Entropy computation is given in eq 2 and eq 3.

$$Entropy(S) = -\sum_{i=1}^{c} p_i \, log_2 \, p_i \tag{2}$$

To compute Gini Impurity, we use the formula:

$$Gini(S) = 1 - \sum_{i=1}^{c} (p_i)^2 \tag{3}$$

where S = sample space and p_i = probability of selecting a datapoint from class i. Finding the characteristic that optimizes information gain or impurity reduction following split is the aim. Until a halting condition is satisfied, the splitting keeps going. Fig 2 illustrate Decision Tree.

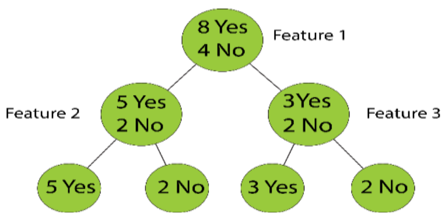

Fig. 2. Representation of Decision Tree

C. K-Nearest Neighbors (KNN)

A non-parametric supervised learning method for regression and classification issues is K-Nearest Neighbors. Since it saves the dataset and uses it when it comes time to classify, it is sometimes referred to as a lazy learner algorithm [13]. This is because it does not learn directly from the training set. The prediction of a data point's classification or grouping is based on its proximity to other surrounding points. In n-dimensional space, the formula for the Euclidean distance between two points, p = (p1, p2,..., pn) and q = (q1, q2,..., qn), is given in eq : 4

$$Distance(p,q) = \sqrt{(p_1 - q_1)^2 + (p_2 - q_2)^2 + \ldots + (p_n - n)^2} \tag{4}$$

In our case, a value of 5 for K gave the least amount of error which is shown in Fig 3.

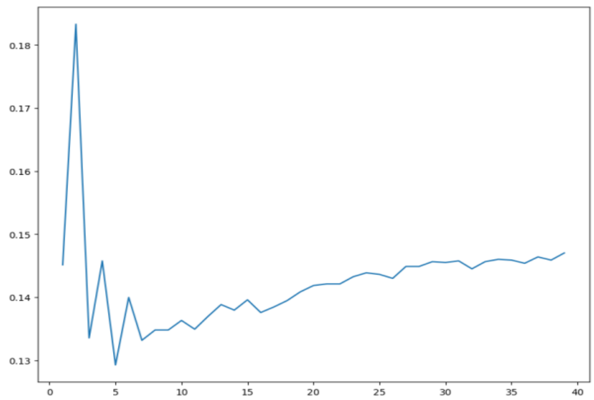

Fig. 3. Distribution of K value vs error rate

D. Support Vector Machine (SVM)

The Support Vector Machine (SVM) algorithm is one type of supervised learning technique used for both regression and classification issues [14]. Finding a hyperplane in an N-dimensional space that maximally divides the various classes in the training data is the fundamental notion behind support vector machines (SVMs).

E. Naïve Bayes (NB)

NB is a classifier based on the Bayes Theorem often used in calculating conditional probabilities [15]. The formula for Bayes' theorem is given in eq: 5.

$$P(A|B) = \frac{P(B|A) \cdot P(B)}{P(A)} \tag{5}$$

$P(A|B)$ denotes the likelihood of class (A) given class (B), $P(B|A)$ denotes the likelihood of class (B) given class (A), and $P(A)$ and $P(B)$ denote the individual likelihoods of class (A) and class (B) respectively.

F. Random Forest (RF)

A supervised learning system called Random Forest is applied to regression and classification problems [16]. It works by creating a size-n random bootstrap sample by selecting n samples at random from the training set and constructing a decision tree. At each node choose d features at random, without substitution. To maximize the information gain, the node can be split using the feature that yields the best split, based on the goal function. Repeat this multiple times and combine each tree's predictions to determine the class label by a majority vote.

G. Ada Boost (AB)

Ada Boost, also known as adaptive boosting, is a supervised learning method that can be used for classification and regression issues. [17]In order to produce a strong classifier, it combines several weak classifiers. It is trained by choosing the training set iteratively according to the previous set's accuracy. In order to provide incorrectly categorized observations a better likelihood of categorization in the following iteration, it gives them a higher weight. High weight will go to the classifier that is more accurate. This procedure repeats until all of the training data fits has been classified or until the chosen number of estimators is reached. The weight for each sample of the first iteration is given in eq: 6

$$Weight\ (x_i) = \frac{1}{n} \tag{6}$$

where x_i = i'th sample, n = number of samples

H. Gradient Boosting (GB)

A supervised learning technique called gradient boosting is applied to regression and classification problems [18]. In order to minimize the loss function, it builds a sequence of weak learners, usually decision trees, and adds them to the model iteratively. Fit the weak learner to the loss function's negative gradient for every iteration. By adding the weak learner to the model, loss function can be minimized by giving it a weight. Continue doing so until a halting requirement is satisfied. The formula for gradient boosting is given in eq 7.

$$F_m(x) = F_{m-1}(x) + \alpha_m h_m(x) \tag{7}$$

I. Voting Classifier (VC)

Voting classifiers are machine learning models that are trained on a set of many models and that predict a class based on the combined output of all classifiers and takes the majority value out of those to anticipate the output class. Fig. 4. illustrates the mechanism of voting classifier.

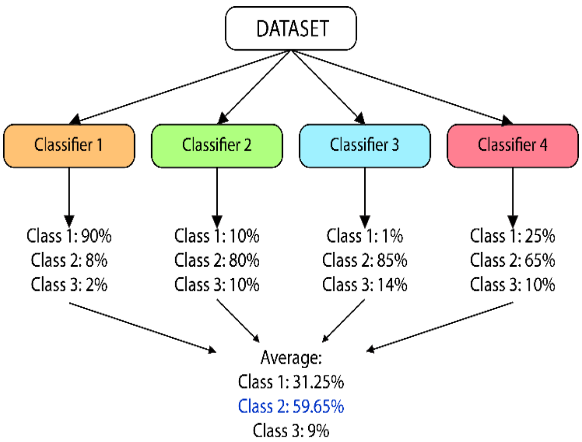

Fig. 4. Representation of Voting Classifier

J. Stacking Classifier (SC)

A machine learning approach called the stacking classifier, sometimes referred to as stacked generalization, combines many base classifiers and employs a meta-classifier to provide final predictions.

Meta-Classifier: A classifier that generates its final prediction using the input of the basis models' predictions.

The benefit of stacking is that it can integrate the advantages of several models, which could result in increased robustness and accuracy [20]. Fig. 5. Illustrates representation of Stacking Classifier.

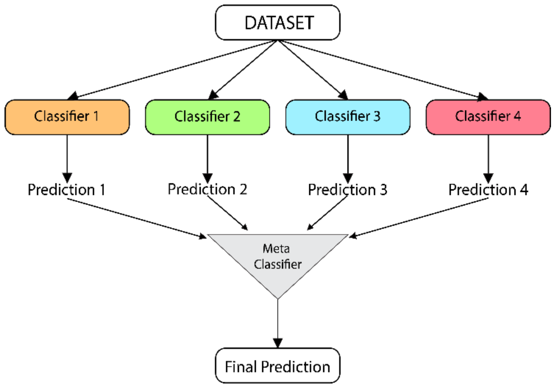

Fig. 5. Representation of Stacking Classifier

V. Implementation And Results

There are three more subsections within this section: A. Hardware and software, B. Experimental Details, C. Results

A. Hardware and Software

The studies were conducted using a laptop running Windows 11 Home OS and outfitted with an Intel i7-1165G7 @ 2.80 GHz processor and 16 GB of RAM. Using Jupyter Notebook and scikit-learn modules, Python 3 was utilized.

B. Experimental Details

The experiment was carried out using 10 classifiers - LR, DT, KNN, SVM, NB, RF, AB, GB, VC, SC. Some specific parameters were used in order to optimize the accuracy of the models. LR was used with parameter multi class set to 'ovr' or one vs rest. Fitting a single classifier per class is the goal of this approach. Every class is fitted against every other class for every classifier. In KNN, the parameter neighbors was set to 5 for least error rate. In SVM, the kernel parameter was set to 'rbf'. RF was used with 100 estimators and a random state of 21 to ensure same results every time the model is run. In VC and SC, we have taken Ada Boost, Gradient Boosting, Random Forest and Decision Tree as the estimators.

C. Results

Fig 6 represents the confusion matrix of stacking classifier. Table I and Fig. 7 show the performance metrics of all classifier models used in this experiment. The values of the table are sorted in increasing order of the accuracy with Stacking Classifier having the highest accuracy of 0.9976. It also has the highest sensitivity, F1 score, MCC, and NPV and has the lowest FNR score. Thus, it is evident that Stacking Classifier works best for classifying this dataset. Fig. 6 shows the confusion matrix for Stacking Classifier.

We notice that Gradient Boosting has a perfect value of 1 in terms of precision but Stacking Classifier has a value of 0.9937 for its precision. This may be due the fact that though Gradient Boosting has better precision, Stacking Classifier is a combination of multiple other classifiers as well, which may have collectively brought the precision of the model down slightly. Since, Ada Boost only has a precision of 0.9 while the other classifiers have a much higher precision, we may be seeing a fall in the precision value of the Stacking Classifier, but it is still much more than enough for an ideal use case.

Naïve Bayes and K-Nearest Neighbors had the worst metrics overall with accuracy and other metrics being less than 80%. Overall, Stacking Classifier has achieved the highest number of maximum values across all metrics discussed above and proves to be the most ideal classifier for our use case.

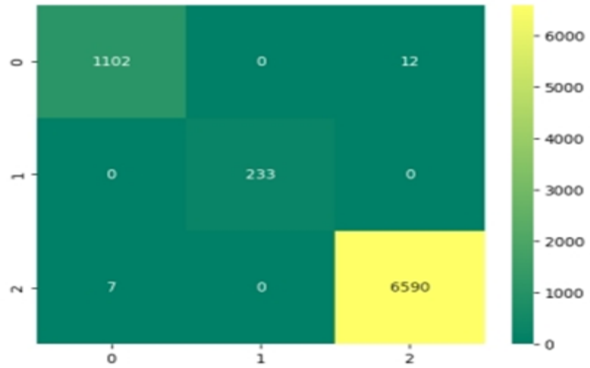

Fig. 6. Confusion matrix of Stacking Classifier

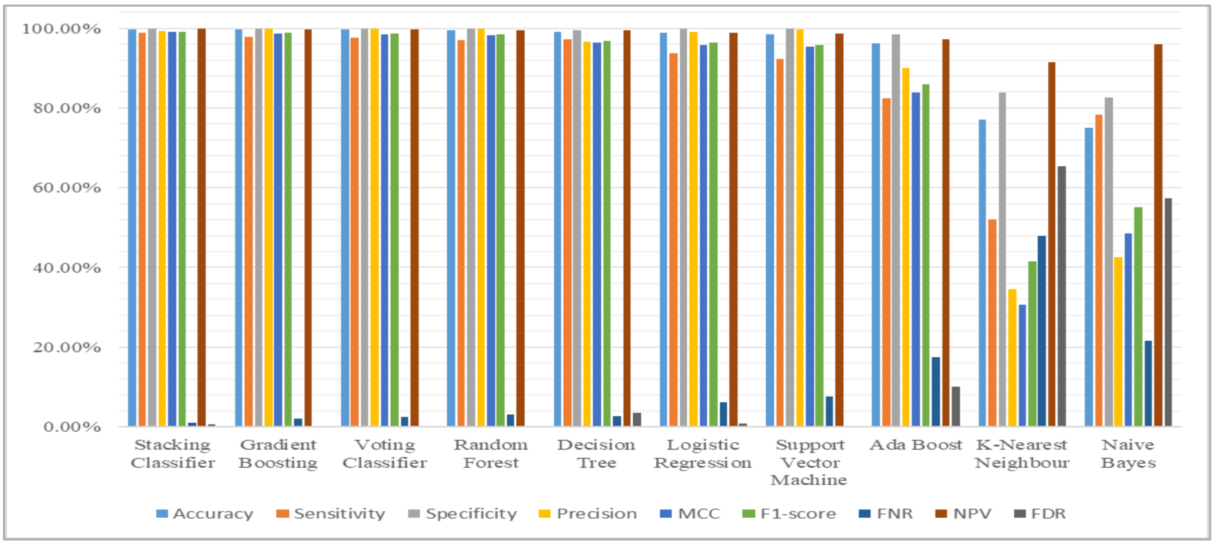

Fig. 7. Comparison of Performance Matrix of all Classifiers

TABLE I. PERFORMANCE METRICS OF ALL CLASSIFIERS

Model	**Accuracy**	**Sensitivity**	**Specificity**	**Precision**	**Matthews Correlation Coefficient (MCC)**	**F1 score**	**False Negative Rate (FNR)**	**Negative Predicted Value (NPV)**	**False Discovery Rate (FDR)**
Stacking Classifier	0.9976	0.892	0.999	0.9937	0.9901	0.9915	0.0108	0.9982	0.0063
Gradient Boosting	0.997	0.9785	1.0	1.0	0.9874	0.9891	0.0215	0.9965	0.0
Voting Classifier	0.9965	0.9758	0.9999	0.9991	0.9853	0.9873	0.0242	0.9961	0.0009
Random Forest	0.9957	0.9695	1.0	1.0	0.9822	0.9845	0.0305	0.9950	0.0
Decision Tree	0.9913	0.9731	0.9943	0.9653	0.9641	0.9692	0.0269	0.9956	0.0347
Logistic Regression	0.9883	0.9372	0.9987	0.9915	0.9583	0.9635	0.0628	0.9898	0.0085
Support Vector Machine	0.9848	0.9237	0.9996	0.9971	0.9535	0.959	0.0763	0.9877	0.0029
Ada-Boost	0.9625	0.8241	0.9851	0.9	0.8398	0.8604	0.1759	0.9717	0.1
K-Nearest Neighbour	0.7705	0.5206	0.8394	0.3459	0.3063	0.4156	0.4794	0.9148	0.6541
Naïve Bayes	0.7497	0.7837	0.8274	0.4254	0.4847	0.5515	0.2163	0.9591	0.5746

VI. Conclusion And Future Work

The financial industry has been transformed by the emergence of machine learning. Predicting client loan repayment behaviour is one of the most important uses of machine learning in finance. Lending institutions can save a lot of money if they can accurately estimate loan payback, which lowers the default risk. This paper covers multiple machine learning algorithms and how they can be used to predict loan repayment behaviour among borrowers.

Financial institutions may use machine learning models to lessen risks, encourage responsible lending practices, and make better-informed lending decisions. Individuals without typical credit records, such students or jobless adults, may find it easier to evaluate their creditworthiness with the use of machine learning. This may result in increased financial inclusion. The study may open the door to more research in this field, including the use of better machine learning techniques, the incorporation of new features, and the application of the models to different specific types of loans. More advanced deep learning models like CNN, RNN etc. may also be utilized to fetch even higher accuracies.

References

[1] Y. Liang, X. Jin and Z. Wang, "Loanliness: Predicting loan repayment ability by using machine learning methods," 2019.

[2] M. Isaeva, "Determinants of loan prepayment and comparison of machine learning approaches," in *2022 IEEE World Conference on Applied Intelligence and Computing (AIC)*, 2022.

[3] A. Sarkar, K. Sai, A. Prakash, G. V. V. Sai and M. Kaur, "A Research Paper on Loan Delinquency Prediction," *International Research Journal of Engineering and Technology,* vol. 8, no. 4, pp. 715-722, 2021.

[4] M. Anand, A. Velu and P. Whig, "Prediction of Loan Behaviour with Machine Learning Models for Secure Banking," *Journal of Computer Science and Engineering (JCSE),* vol. 3, pp. 1-13, 2022.

[5] A. Mueankoo, M. Eso and S. Musikasuwan, "Performance of a Loan Repayment Status Model Using Machine Learning," in *International Conference on Electrical Engineering/Electronics, Computer, Telecommunications and Information Technology, ECTI-CON*, 2022.

[6] C. Han, "Loan Repayment Prediction Using Machine Learning Algorithms," in *University of California, Los Angeles ProQuest Dissertations Publishing*, 2019.

[7] A. Sarkar, "Machine learning techniques for recognizing the loan eligibility," *International Research Journal of Modernization in Engineering Technology and Science,* vol. 3, no. 12, pp. 1135-1142, 2021.

[8] Y. Diwate, P. Rana and P. Chavan, "Loan Approval Prediction Using Machine Learning," International Research Journal of Engineering and Technology (IRJET), vol. 8, no. 5, pp. 1741-1745, 2021.

[9] M. Madaan, A. Kumar, C. Keshri, R. Jain and P. Nagrath, "Loan default prediction using decision trees and random forest: A comparative study," in IOP Conference Series: Materials, 2020.

[10] "Loan Approval Analysis," *www.kaggle.com*. https://www.kaggle.com/datasets/arbazkhan971/loan-approval-analysis

[11] J. David W. Hosmer, S. Lemeshow and R. X. Sturdivant, Applied logistic regression, John Wiley & Sons, 2013.

[12] S. Suthaharan, "Decision Tree Learning," in *Machine Learning Models and Algorithms for Big Data Classification*, Springer, Boston, MA, 2016, pp. 237-269.

[13] M. L. Zhang and Z. H. Zhou, "ML-KNN: A lazy learning approach to multi-label learning," Pattern Recognition, vol. 40, no. 7, pp. 2038-2048, 2007.

[14] V. Cherkassky and Y. Ma, "Practical selection of SVM parameters and noise estimation for SVM regression," Neural Networks, vol. 17, no. 1, pp. 113-126, 2004.

[15] F.-J. Yang, "An Implementation of Naive Bayes Classifier," in International Conference on Computational Science and Computational Intelligence (CSCI), 2018.

[16] L. Breiman, "Random Forests," Machine Learning, vol. 45, p. 5–32, 2001.

[17] Y. CAO, Q.-G. MIAO, J.-C. LIU and L. GAO, "Advance and Prospects of AdaBoost Algorithm," Acta Automatica Sinica, vol. 39, no. 6, pp. 745-758, 2013.

[18] C. Bentéjac, A. Csörgő and G. Martínez-Muñoz, "A comparative analysis of gradient boosting algorithms," Artificial Intelligence Review, vol. 54, p. 1937–1967, 2020.

[19] U. K. Kumar, M. S. Nikhil and K. Sumangali, "Prediction of breast cancer using voting classifier technique," in International Conference on Smart Technologies and Management for Computing, Communication, Controls, Energy and Materials (ICSTM), 2017 .

[20] E. S. Lee, "Exploring the Performance of Stacking Classifier to Predict Depression Among the Elderly," in IEEE International Conference on Healthcare Informatics (ICHI), 2017.